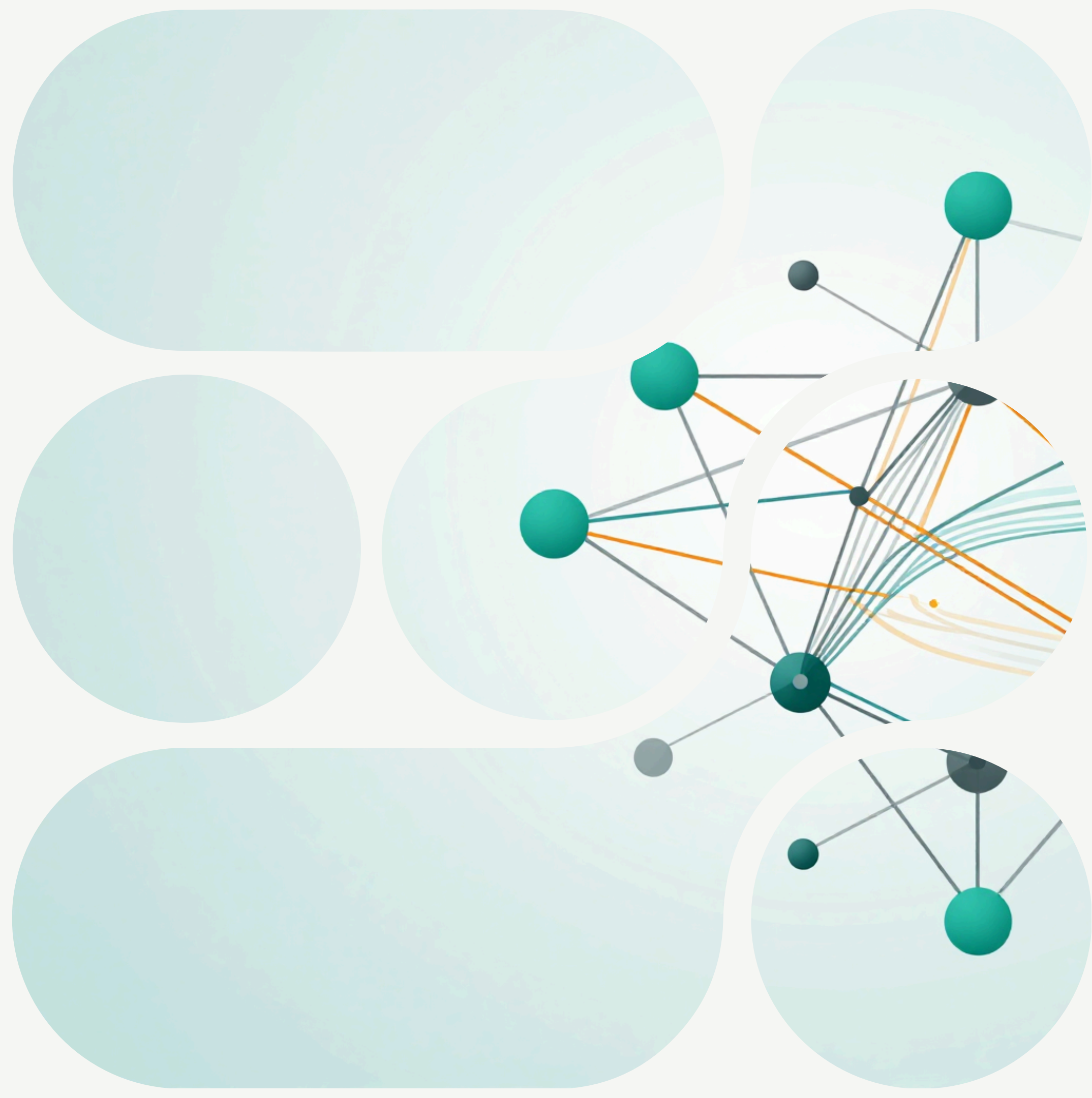


Clojure

21 de septiembre de 2026

PRESENTACIÓN TÉCNICA



Orígenes Históricos

En 2007, Rich Hickey creó Clojure como una respuesta a los desafíos de programación en la Java Virtual Machine (JVM). El lenguaje se diseñó para abordar problemas de concurrencia y simplicidad que los desarrolladores enfrentaban con Java.

Clojure elimina la mutabilidad y el uso de null, problemas comunes en lenguajes imperativos. Al ser un Lisp moderno, ofrece una sintaxis expresiva para programación funcional, ideal para entornos concurrentes.



Historia

2007

Origen: Lanzamiento público por Rich Hickey como un proyecto independiente para solucionar los dolores del desarrollo comercial.

Madurez (v1.0): Primera versión estable, adopción temprana en la industria y primeros libros técnicos.

2009

2011

Expansión y Filosofía: Nace ClojureScript (para navegadores) y Hickey presenta "Simple Made Easy", definiendo la filosofía del lenguaje.



Historia

2012

Ecosistema: Lanzamiento de la base de datos inmutable Datomic y fundación de Cognitect para respaldar el núcleo del lenguaje.

Respaldo Corporativo: El gigante tecnofinanciero Nubank adquiere Cognitect, garantizando el financiamiento y desarrollo de código abierto a largo plazo.

2020



Lisp sobre la JVM y sintaxis

```
(+ 10 20)
```

(FUNCIÓN ARGUMENTOS)

En Lisp, la función aparece primero y después sus argumentos.

```
(println "Hola")
```



Clojure pertenece a la familia Lisp y se ejecuta sobre la Java Virtual Machine (JVM).

No modificamos lo anterior: creamos una nueva versión

VERSIÓN ORIGINAL

(def lista [1 2 3])

LISTA

[1 2 3]

NUEVA VERSIÓN

(def nueva (conj lista 4))

NUEVA

[1 2 3 4]

LISTA

[1 2 3]



NUEVA

[1 2 3 4]

No modificamos lo anterior; creamos una nueva versión y reutilizamos lo que no cambió. Persistente no significa guardar en disco; significa conservar disponibles las versiones anteriores.

atom cambia uno; ref + dosync coordina varios

ATOM

```
(def contador  
  (atom 0))  
(swap! contador  
  inc)
```

ESTADO
INDEPENDIENTE

REF + DOSYNC

```
(dosync  
  (alter cuenta-a  
    -20)  
  (alter cuenta-b  
    +20))
```

CAMBIOS
RELACIONADOS
DENTRO DE UNA
TRANSACCIÓN

STM · SOFTWARE TRANSACTIONAL
MEMORY

UN CAMBIO O UNA COORDINACIÓN

ATOM

Cambio independiente.

REF + DOSYNC

Cambios relacionados /
transacción.

Transducers componen transformaciones sin colecciones intermedias

01

ENTRADA

[1 2 3 4 5 6]

02

FILTER

números pares

03

MAP

×10

SALIDA

[20 40 60]

Componen transformaciones sin depender de colecciones intermedias.

Clojure → Lisp sobre JVM → datos inmutables → estructuras persistentes → atom / ref + dosync → [transducers](#)

Clojure combina la sintaxis de Lisp con un modelo funcional para manejar datos, estado y transformaciones.

Sistema de Tipos y Concurrency

- Tipado dinámico con chequeos en tiempo de ejecución
- Runtime robusto basado en JVM
- Concurrency basada en atom y ref/dosync
- Programación concurrente segura y expresiva
- sin bloqueos pesados

atom

actualización atómica simple

vs

ref/dosync

transacciones STM para operaciones complejas

Demo en Vivo



Hola Paradigma (imprimir 1..10)
clj
**user=> (doseq [i (range 1 11)]
(println i))**



Segundo ejemplo idiomático

**user=> (->> (range 1 11)
(filter even?) (map #(* % %)))**

<https://asciinema.org/a/enYsUIEDs9icN9HW>

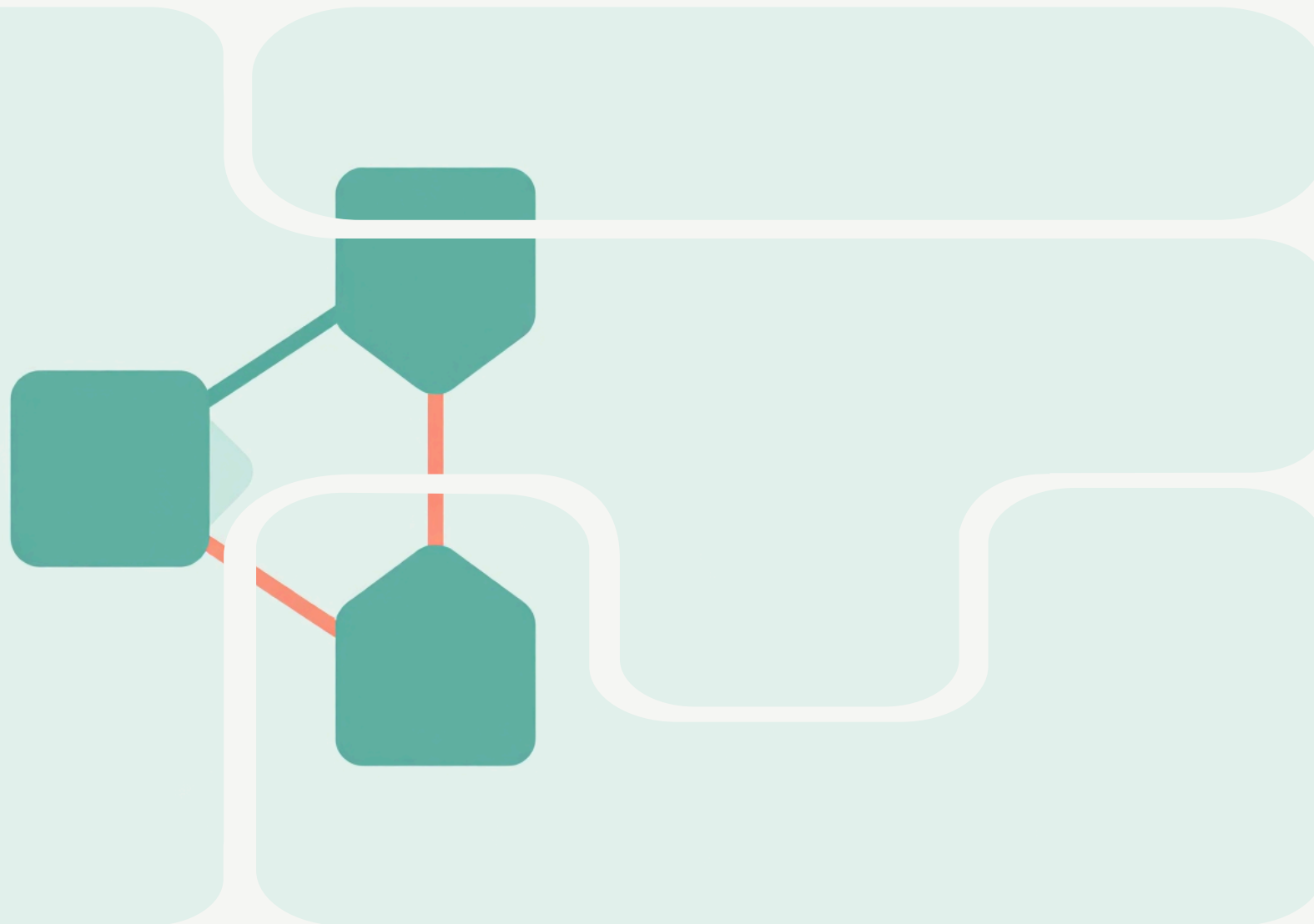
Caso Real: Nubank



Nubank, el banco digital más grande de América Latina, construyó su plataforma con Clojure y la base de datos Datomic.

"Un banco maneja dinero y transacciones, donde un dato que cambia a escondidas puede causar errores graves."

Conclusión

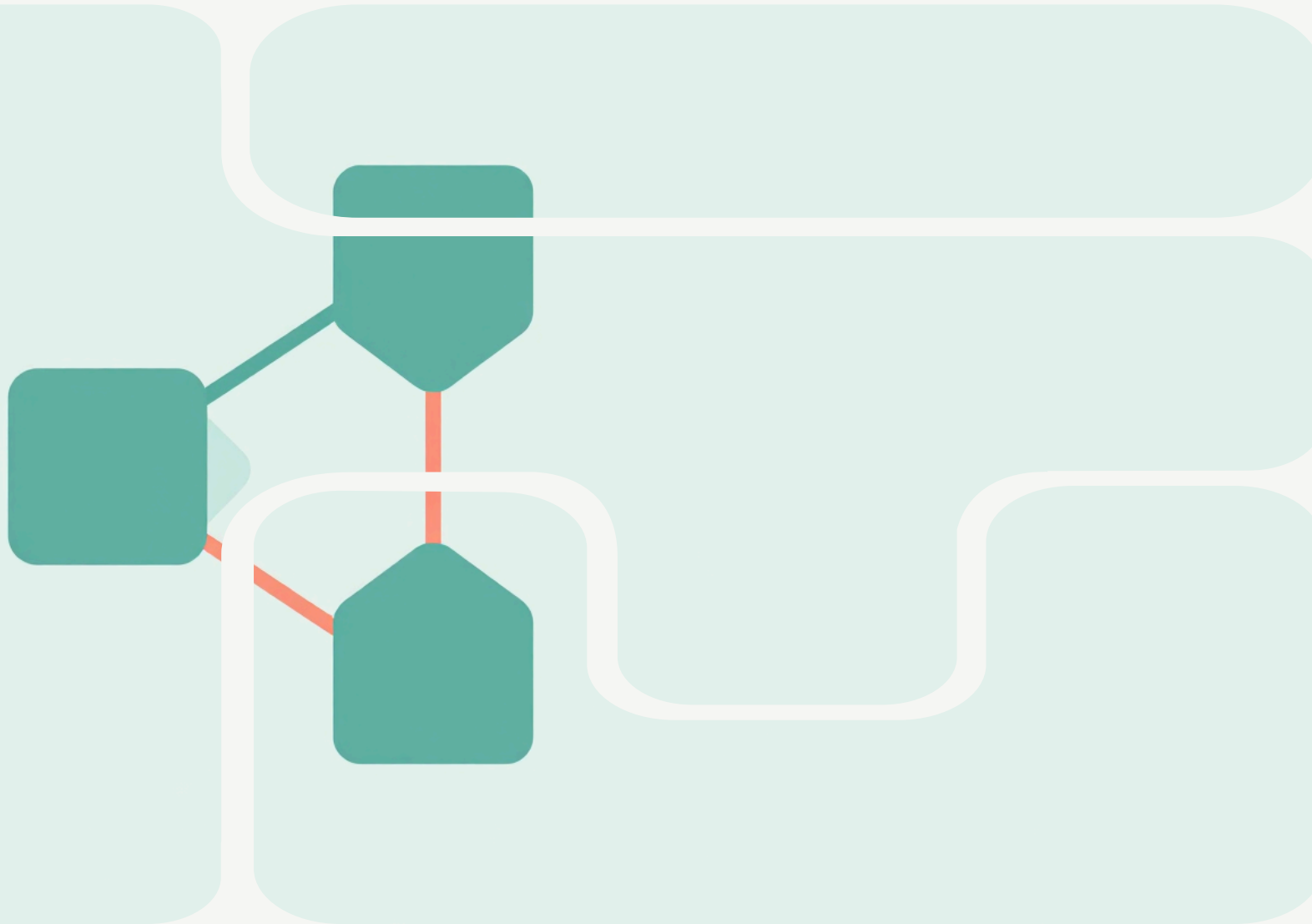


Clojure es un Lisp moderno: hereda la sintaxis de paréntesis y la idea de código como datos de Lisp y Scheme.

Clojure no es un lenguaje nuevo desde cero.

Se creo principalmente pensando en:
Los errores de concurrencia

Referencias



Clojure

Clojure es un lenguaje de programación de propósito general dialecto de Lisp. Hace un énfasis especial en el paradigma funcional, con el objetivo de eliminar la...

 Wikipedia



Clojure

Clojure is a dynamic, general-purpose programming language, combining the approachability and interactive development of a scripting language with an efficient and...

 clojure.org

